

CSI247:
Data Structures

Lecture 12 – Generics

Department of Computer Science
B. Gopolang (247-275)

Previous Lesson

- OOP
  - Inheritance
    - Super class vs sub class
    - Representing inheritance in UML diagrams
  - Encapsulation
  - Polymorphism
  - Abstraction
    - Abstract classes
    - Interfaces
      - Representing interfaces in UML diagrams



Today …

Generics



Lecture Objectives

- By the end of this lesson you must be able to:
  - Understand the concept of Generics
  - Understand advantages of Generics
  - Understand syntax of Generics
  - Create a small Java program using Generics

1. Introduction

- Rem:
  - All classes are descendants of class Object
    - They inherit from the root class
  - E.g.
    - String, Dog, Shape, etc…

- We can assume all these objects of different types are "things"
  - Then treat them as if they belong to the same reference type

UNIVERSITY of BOTSWANA

Generics: OOP concept that refers to parameterized types

- Allows for the creation of a single class, method or interface that works with different types of objects
  - Implementation is independent of class type
    - Allows for creation of entities that can work with different reference data types
- Allows for various object types to be passed as parameters to
  - Methods
  - Classes
  - Interfaces

Advantages of Generics

- Code Reuse:
  - Write code once, then reuse for any type
- Type safety
  - We store only one type, the parameter!
- No need for type casting
  - Rem:

```
int x = (int) 7.8;

Object ob = new (Object) sh // sh is a Shape object
```

- Errors caught during compilation than at run-time

Types/Levels of Generics in Java

a) Generic class

- A class that accepts a type parameter
- Its class header has a type parameter section
  - Added after the class name, inside angle brackets
    - <T>
- Such a class is aka **parameterised class**
  - i.e.: classes that accept type parameter(s) during object creation

- Syntax of a generic class

```
class Test <T> {
// class Test is a generic class
    // class contents
}
```

- Note
  - Test: name of the generic class
  - T: class' parameter

- Example

```
class GenTest <T> {
    private T obj;  // instance variable
    void modify (T obj) {
        this.obj = obj; //setter ?
    }
    T get(){
     return obj;    // getter ?
   }
}
```

- **Dissecting the Generic class Example:**
  - < > : angle brackets
    - implies type parameter passing during object creation
    - **Only reference data types!**
  - **T**: A formal type parameter for the generic class
    - A placeholder for any valid type parameter
    - Can be used anywhere inside the class, including:
      - As a method parameter & a method return type
      - Except for allocating memory
        - E.g. T[] arr = new T[20];

- 4 occurrences of T in class GenTest:

a) *class GenTest <T> // Read as "GenTest of T"*

- *GenTest accepts only 1 type parameter, called* ***T***
  - **T:** a type variable placeholder
    - Replaced by the actual type argument during object creation

b) *private T obj* : instance variable's data type is T

c) *void modify (T obj)* : method accepts 1 object, of type T

d) *T get()*: method returns a value of type T

- **Syntax for creating objects** of a generic class:

  BaseType <Type> obj-name = new BaseType <Type>();

  - BaseType: Generic class' name & constructor name
  - Type: Class type parameter, e.g. Integer, String, etc
    - Types on L and R side of = must be identical!
    - DO NOT OMIT () at the end
  - obj-name: name of object to create

Note

- A specific class type is placed inside angle brackets during object creation

- Using class GenTest in a program:

Tells Java that we will be dealing with Integer type

```
class TestingGenTest{
    public static void main(String args[]) {
        GenTest<Integer> a = new GenTest<Integer>();
        a.modify(17);   // method accept an Integer
        System.out.println(a.get()); // method returns an Integer
        // can a.modify("CSI247"); compile? Justify.
    }
}
```

- GenTest<Integer> is read as "GenTest **of Integer**"

- Generic class can also accept more than 1 type parameter
  - Separate type parameters with a comma (,)
  - E.g.:

```
class Test <T, U> {     // T & U: 2 type parameters
        // class contents
}
```



- Type parameter naming convention
  - Different letters used for different type parameters
  - Most common ones are:
    - T: Type
    - E: Element
    - K: Key
    - N: Number
    - V: Value

Bounded Types

- We can restrict types of objects the class can manipulate
  - E.g. a generic class might be specific to numbers only
    - This will include Integer, Double, Float, Long, etc

Bounded types:
Refers to restricted type parameters

- **How to?**
  - Add **extends** keyword after the parameter type
  - Followed by the **name of the super class** for the acceptable sub classes

- Example

```
class Nums <T extends Number > {
  // …
}
```

- Note: **Number** is a super class for Integer, Double, Float, etc
- Hence these are all valid statements

```
Nums <Double > n1 = new Nums <Double > (new Double (5.5 ) );
Nums <Integer> n2 = new Nums <Integer> (new Integer (2 ) );
```

- How about this?

```
Nums <String> n3 = new Nums <String> (new String("Hello") );
```

b) Generic method

- Method that accept type parameters (arguments)
  - Type parameters
    - Placeholders for the method's type(s) of arguments
    - Can be used as return types for non-void methods
  - Scope of argument(s) is limited to the method
- Method's body is declared like in any other method
- Works with both static and non-static methods

How to?

- Add <E> before method's return type

- Example: generic method that prints array elements

```
public class GenericMethod {

    …

    public static <E> void printArray( E[] elements) {
        for (E element : elements) {   // for each element in the array
                System.out.printf("%s ", element);
        }
        System.out.println();
    }
```

```
// Testing the generic method
public static void main(String args[]) {
        Integer[ ] numbers = { 2, 4, 5, 7, 8 };
        Double[ ] rates = { 1.0, 1.5, 2.5, 3.5 };
        System.out.println("Integer Array has : ");
        printArray(numbers);        // accept Integer array
        System.out.println("\n Double array has : ");
        printArray(rates);          // accept Double array
   }
}
```

- **Note**:
  - Like the class, a generic method can accept one or more type parameters
    - Parameters are separated by commas
  - Methods can use class' type parameters
  - They can also introduce additional type parameters, if there is a need
    - These parameters can be used like the type parameters in the method header

- Exercises

a) Type all examples given in the handout
   - Compile them, and fix all errors if there are any
   - Test them and observe the output

b) Regarding the last one, modify main method to also print the following arrays
   - Characters
   - Strings
   - Floats

c) Generic interface

- Similar to a normal interface
  - Except, it MUST include a parameter type
- Example:

```
interface BigAndSmall<T extends Comparable<T> > {
    // has 2 abstract methods: small() & big()
   T small();
   T big();
}
```

- Class implementing the interface must also be generic
- Otherwise
  - Compile-time error

- Example:

```
// Generic Interface
public interface GenI <T> {
    void modify(T val);
    public T get();
}
```

```
// Generic class
public class GenC <T>
implements GenI <T>{
  T val;
  public void modify (T val) {
       this.val= val;
  }
  public T get() {
      return val;
  }
}
```

```
// Tester class
public class GenCTester{
public static void main(String[] args) {
    GenC<Integer> n = new
    GenC<Integer>();

     n.modify(190);
     System.out.println("Num: "+
     n.get());
  }
}
```



6. Summary

- Generics
- Advantages of using generics
- Types of generic
  - Generic classes
  - Generic methods
  - Generic interfaces
- Using bounded types
- Programming using generics

Exercises

a) Type all code segments provided in this handout into complete classes
   - Compile all files
   - Run your tester files

b) Make changes in your Tester class in relation to the type parameters passed
   - Any observation?
   - What is your justification for the observation(s)?



Next lesson…

Java Collections Framework

Q & A

